AeroPulse AI

Predict early. Maintain smarter. Fly ready.

AeroPulse AI is a web application that helps aircraft maintenance teams answer three questions: **which engine needs attention, can we trust the computer's warning, and what is the best plan with the technicians and spare parts we actually have?** It is an **advisory** tool — it suggests, people decide.

15.7	90%	3	9	93
cycles avg. error on NASA test engines	confidence ranges, measured 89–90%	trust states for every prediction	screens, overview to audit log	automated tests, all passing

THE PROBLEM, IN PLAIN WORDS

Planes wait on the ground for the wrong reasons

Military air fleets lose availability because maintenance is mostly **reactive** — things are fixed after they break — and the information needed to act early lives in separate places: engine sensor data, paper or digital technical logs, spare-part stores, and technician rosters.

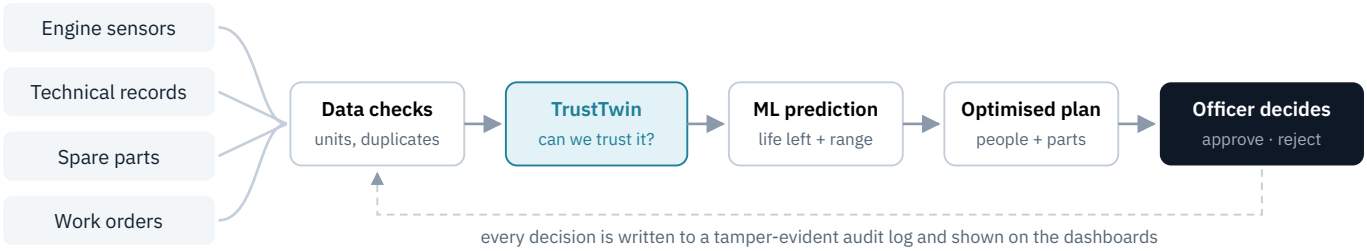
When these don't talk to each other, faults are spotted late, aircraft wait for parts nobody ordered, and work is scheduled without knowing which job matters most.

WHAT WE BUILT

One platform, five jobs

1 · Collect	Imports sensor data, maintenance records, spare parts and work orders; rejects bad or conflicting rows.
2 · Predict	A machine-learning model estimates each engine's remaining life, with a confidence range.
3 · Check trust	**TrustTwin** tests the input data and refuses to give a number when it can't be trusted.
4 · Plan	An optimiser builds a work schedule that respects skills, shifts, parts and task order.
5 · Approve	An officer approves or rejects with a reason; every action is recorded.

HOW IT WORKS, END TO END



Maintenance officer

Reviews evidence; approves or rejects with a reason.

Technician

Sees assigned jobs; marks them started or done.

Analyst

Explores data and runs what-ifs; cannot approve.

Administrator

Manages users, roles and thresholds.

> Important: AeroPulse never decides whether an aircraft may fly. It is a prototype trained on public, simulated engine data, and every aircraft, part and person inside it is fictional.

THE DATA AND THE MACHINE-LEARNING MODEL

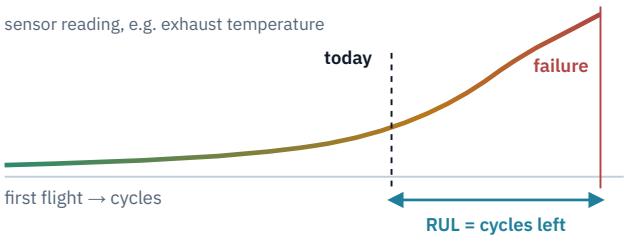
Teaching a computer how engines wear out

Where the data comes from

We use NASA's public **C-MAPSS** dataset ("Turbofan Engine Degradation Simulation"). NASA simulated jet engines running flight after flight until they wore out, recording **21 sensors** — temperatures, pressures, fan and core speeds — once per flight *cycle*. We use the subset **FD001**: 100 engines to learn from, 100 separate engines to test on, one flight condition and one kind of wear (the high-pressure compressor).

Downloaded from NASA's official archive and fingerprint-checked (SHA-256) so anyone can reproduce it. It is simulated commercial-engine data — useful for building and testing the method, not proof about military aircraft.

What the model predicts: Remaining Useful Life



How the model learns — in four steps

STEP 1 · CLEAN
Drop 9 sensors that never change, keep 12 useful ones.

STEP 2 · DESCRIBE
For each sensor: recent average, how much it wobbles, and its trend — using only past flights, never the future.

STEP 3 · SPLIT FAIRLY
Whole engines go into separate groups, so the model is always tested on engines it has never seen.

STEP 4 · COMPARE
Try a simple average, a straight-line model and LightGBM; keep whichever is most accurate on unseen engines.



NASA's 100 training engines, split by engine (never by row), plus NASA's separate 100-engine test set for the official score

The chosen model: LightGBM

LightGBM builds **hundreds of small decision trees**. Each tree asks simple questions ("is the compressor temperature trend above X?") and each new tree fixes the mistakes of the ones before it. It is fast, runs on an ordinary laptop CPU, and — importantly — its reasoning can be explained exactly.

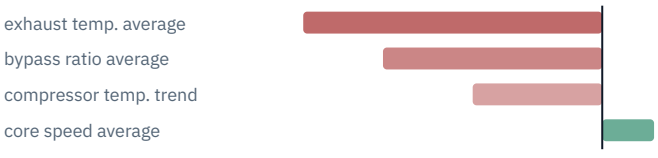
MODEL (NASA TEST SET, 100 ENGINES)	AVG. ERROR*	TYPICAL ERROR
LightGBM (chosen)	**15.7** cycles	11.5 cycles
Straight-line model (Ridge)	21.5	17.0
Always guess the average	43.0	35.9

*RMSE (root-mean-square error) and MAE (mean absolute error). When an engine has ≤ 50 cycles left — the moment that matters — the error drops to 8.2 cycles.

Not just a number — a range

A single guess like "13 cycles left" hides uncertainty. AeroPulse says **"between 0 and 26 cycles, with 90% confidence"**, using a method called *conformal prediction*. We checked it on engines the model never saw: the true answer fell inside the range **89–90%** of the time — the confidence is measured, not assumed.

Why did it say that? — SHAP



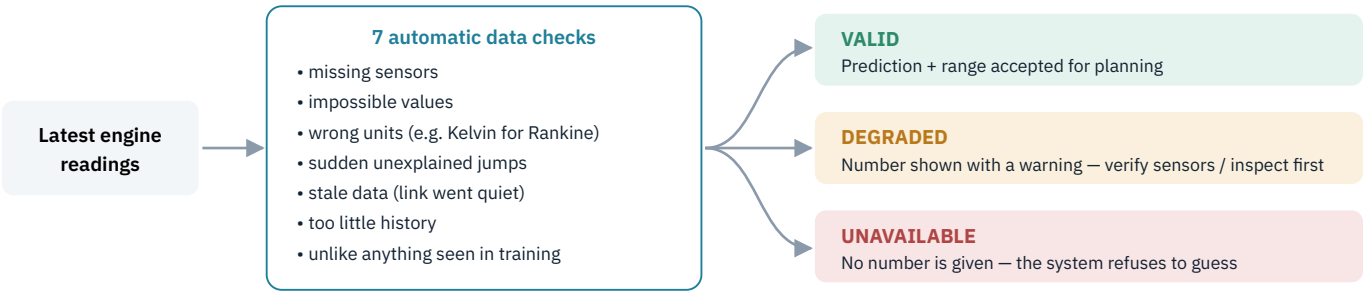
Every prediction lists which sensor trends pulled the estimate down (red) or up (green), in cycles.

> We also benchmarked NASA's three harder subsets (several flight conditions, two kinds of wear). LightGBM still beat the simpler models everywhere (errors 17–27 cycles), but the app uses the FD001 model, where results are strongest.

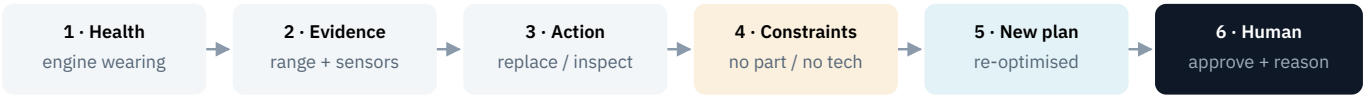
WHAT MAKES AEROPULSE DIFFERENT

TrustTwin: “Can we trust this — and what is the safest plan?”

Most predictive-maintenance projects stop at a prediction. Real maintenance is messier: sensors fail, data arrives late, spare parts run out and technicians go on leave. **AeroPulse TrustTwin** is the layer that handles that reality before anyone acts.



THE WHAT-IF SIMULATOR, AS A STORY



Each step is computed for a “before” and an “after” version, so the officer sees exactly what changed and why.

Engine AP-ENG-01 is wearing out

The model predicts about 13 cycles left (range 0–26). All checks pass → VALID. Recommendation: plan a module replacement.

The spare part is out of stock

The optimiser re-plans with the real stock and delivery date. If the part can't arrive in time, it says so — it never hides the conflict.

Two sensors stop reporting

TrustTwin downgrades to DEGRADED and suggests an inspection first. The officer approves with a reason; it's logged.

The planner: a puzzle solver for maintenance

We use **Google OR-Tools CP-SAT**, a constraint solver. We give it rules that can never be broken — the right skill, working shifts and leave, parts in stock or arriving, tasks in the correct order, one job per aircraft at a time — and a goal: **finish the most urgent work first with the fewest late jobs**. A separate checker re-verifies every plan, and a simple rule-based scheduler is kept for comparison. Jobs that can't fit are listed with the exact reason.

Typical demo vs AeroPulse

TYPICAL PREDICTIVE DEMO	AEROPULSE AI
One number	Number + 90% range + trust state
Uses bad data silently	Flags it, or refuses to predict
Dashboard only	Real plan with people + parts limits
“The AI decides”	Officer approves; audit trail
Claimed accuracy	Measured, reproducible results

> Honest result from our fleet simulation (reactive vs predictive maintenance, same events): predictive planning removed all 20 unexpected engine failures, but overall availability barely moved (92.0% → 92.2%) because early replacements use up spare parts. We report this exactly as computed — it is a simulation, not a promise about real fleets.

Advisory only

Never an airworthiness or flight decision; people approve.

Honest data

Public simulated engines; not validated on military aircraft.

Fictional fleet

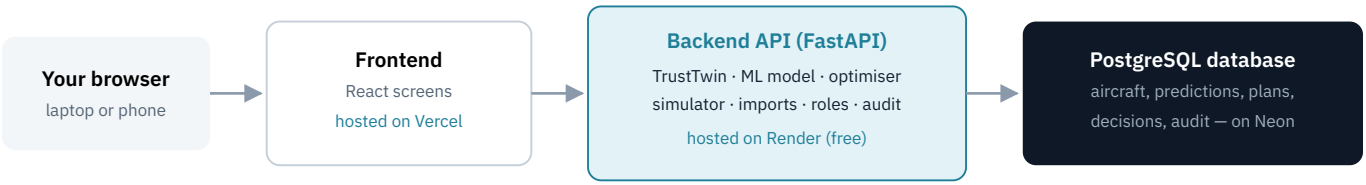
Aircraft, people and parts are invented; thresholds are prototypes.

No made-up numbers

Every figure on screen is computed from data or labelled.

HOW IT IS BUILT

Architecture and technology stack



UptimeRobot checks the API every 5 minutes · the same system also runs fully offline on one machine with Docker + local PostgreSQL

LAYER	TECHNOLOGY	WHAT IT DOES, IN SIMPLE WORDS
Screens	React 19 + TypeScript, Vite	The nine interactive pages; TypeScript catches mistakes early; Vite builds the app.
	Tailwind CSS, IBM Plex fonts, lucide icons	The calm “operations console” look; fonts/icons bundled, nothing loads from the internet.
	TanStack Query, React Router, Recharts	Fetches and caches live data, page navigation, and all charts.
Server	Python 3.11, FastAPI, Pydantic	The “brain” the screens talk to; validates every input; auto-generated API docs.
	SQLAlchemy + Alembic	Safe database access; Alembic versions every change to the database structure.
	bcrypt, JWT, role checks, optional SSO (OIDC)	Secure logins, four roles enforced by the server, optional organisation single sign-on.
Machine learning	pandas, NumPy, LightGBM, joblib	Prepares sensor data; trains and runs the remaining-life model.
	scikit-learn, SHAP (training only)	Baselines for fair comparison and per-sensor explanations (laptop only).
Planning	Google OR-Tools CP-SAT	Best feasible schedule under hard rules (skills, shifts, parts, order).
Data	PostgreSQL (Neon / local Docker)	Stores everything — in the cloud or on a disconnected on-premise machine.
Quality	pytest, Vitest, Playwright, ruff, mypy	93 automated tests (model, safety, planner, roles, screens, browser) + code checks.
Hosting	Vercel · Render · Neon · UptimeRobot, Docker	Free-tier public demo with a 5-minute health ping; Docker for offline use.

The nine screens

Command overview · Aircraft fleet · Aircraft health (sensor charts, engine schematic, evidence) · Predictive intelligence (real model scores) · Maintenance planner (timeline chart) · Spare parts · **TrustTwin** workspace · Reports & audit (CSV / PDF export) · Settings & data imports.

Path to real-world use

- **1 · Prototype** — public NASA data (done)
- **2 · Pilot data** — authorised, non-classified
- **3 · Independent validation** of model + data
- **4 · Security review** + on-premise install
- **5 · Supervised pilot** with real maintainers
- **6 · Wider adoption** — only after acceptance

Live demo
aero-pulse-ai-kohl.vercel.app

Source code
github.com/sahilkarande0918-cmd/AeroPulse-AI

Dataset
NASA PCoE · C-MAPSS FD001 (Saxena et al., 2008)